TECHNICAL PLAN 2026

WhatsApp CRM Bot

A practical plan for building a WhatsApp bot that collects client information, delivers PDFs, manages scheduling, tracks deals and expenses via slash commands, and feeds everything into a CRM dashboard for filmmakers and photographers.

Stack: Node.js + Supabase + Lovable/Next.js

Date: May 2026

Z.AI

Table of Contents

1. Project Overview

This plan outlines the development of a WhatsApp bot that serves as both a client-facing entry point and an internal operations tool for filmmakers and photography professionals. Clients interact with the bot to share their information, receive documents (contracts, quotes, shot lists), and schedule sessions. Team members use slash commands inside WhatsApp to quickly create deals, register briefings, track expenses, and check project status. All data flows into a central CRM/dashboard, giving the professional a single place to manage clients, calendar, documents, and finances.

The target user is a solo filmmaker or photographer, or a small studio, who needs a lightweight CRM without the overhead of enterprise tools. The developer (you) is a JavaScript developer, so the entire stack is JS-centric: Node.js backend, Supabase for database and edge functions, and Lovable for the dashboard frontend (with Next.js as the recommended path for the production dashboard beyond MVP).

What	Description
Product	WhatsApp bot + CRM dashboard for filmmakers/photographers
Users	Filmmakers, photographers, small production studios
Client entry	WhatsApp conversation (bot collects info, sends PDFs, offers scheduling)
Team entry	WhatsApp slash commands (/novoDeal, /briefing, /despesa, /status)
Pro entry	Dashboard (view clients, manage calendar, send PDFs, track finances)
Tech stack	Node.js, Supabase, WhatsApp Cloud API, Lovable/Next.js, Cal.com API

Table 1: Project Overview

2. Core Features

2.1 Client Intake via WhatsApp

When a new client messages the business on WhatsApp, the bot initiates a structured intake flow. Using WhatsApp interactive messages (quick reply buttons and list menus), the bot collects the essential information a filmmaker or photographer needs before a shoot: client name, event type (wedding, corporate, portrait, product, etc.), preferred date, location, and any special requests. Each response is stored in the Supabase database under a new client record, linked to a conversation thread. The professional sees the new client appear in their dashboard in real-time via Supabase Realtime subscriptions.

Example flow: (1) Client sends "Hi" or clicks a WhatsApp ad. (2) Bot replies with quick reply buttons: "Book a Shoot" / "Get a Quote" / "Talk to Us". (3) Client taps "Book a Shoot" and the bot asks event type via list message. (4) Bot asks date, location, and special notes using sequential prompts. (5) Bot confirms all details and creates a client record + calendar slot hold. (6) Dashboard updates instantly with the new client and pending booking.

2.2 PDF Delivery via WhatsApp

Sending PDFs through WhatsApp is a native capability of the Cloud API. The bot can send documents up to 100 MB in size with a filename and optional caption. For a filmmaker/photographer CRM, the key PDF scenarios are: sending contracts for digital signature confirmation, delivering price quotes and packages, sharing shot lists or mood boards before a shoot, and delivering final deliverables (invoice, receipt, gallery link as PDF). The PDFs are generated server-side using a library like pdfkit or puppeteer (for HTML-to-PDF), stored in Supabase Storage, and sent as a document message through the Cloud API.

From the dashboard, the professional selects a client, chooses a document template (contract, quote, invoice), fills in the variable fields, and clicks "Send via WhatsApp." The dashboard calls a Supabase Edge Function that generates the PDF, uploads it to Supabase Storage, and sends it as a WhatsApp document message. The client receives the PDF directly in their chat.

2.3 Calendar and Scheduling

Calendar integration is critical for filmmakers and photographers who juggle multiple shoots, client meetings, and editing sessions. The recommended approach is to integrate with Cal.com (open-source scheduling infrastructure) which provides a developer-friendly API, customizable booking pages, and webhook notifications. When a client selects a date through the WhatsApp bot, the backend creates a pending booking in Cal.com. The professional sees it in their dashboard and can confirm, reschedule, or decline. Status changes are pushed back to the client via WhatsApp template messages.

Cal.com offers a self-hosted option (important for data control) and a cloud version with a free tier. Its API supports creating event types, managing availability, and handling timezones natively. The alternative is building a simple calendar table in Supabase and rendering it in the dashboard, which is faster to implement but lacks features like timezone handling, conflict detection, and client-facing booking pages.

Option	Pros	Cons	Effort
Cal.com	Full scheduling, timezones, booking pages, webhooks	External dependency, API learning curve	Medium
Supabase table	Simple, no external dependency, full control	No timezone logic, no booking pages, manual conflict handling	Low

Google Calendar	Familiar, existing ecosystem	OAuth complexity, rate limits, not self-hosted	Medium

Table 2: Calendar Integration Options

2.4 Dashboard / CRM

The dashboard is where the professional manages everything: client list, conversation history, calendar, document templates, financial overview, and analytics. The recommended approach is a two-phase strategy: start with Lovable for the MVP, then migrate to a custom Next.js application for the production dashboard. This gives you speed to market without sacrificing long-term control and scalability. Both dashboards connect natively to the same Supabase backend, so migration is a frontend-only change.

2.5 Bot Slash Commands (Team Operations)

Beyond client-facing flows, the bot serves as a rapid data-entry tool for the team. Instead of opening the dashboard to log a new deal or register an expense, team members type slash commands directly in WhatsApp. This is especially valuable for filmmakers and photographers who are often on-set or on-location and need to capture information quickly from their phones. The bot parses the command, validates the input, writes to Supabase, and confirms the operation. The dashboard reflects the changes in real-time.

A. /novoDeal - Create a New Project

When a team member types `/novoDeal` followed by a client name and project type, the backend creates a new Contact record and a Deal record in Supabase. The Deal is the central entity that links all other data: briefings, expenses, documents, and calendar events. The bot returns a confirmation with the project ID, which is used in all subsequent commands. This replaces the traditional "open CRM, click New Deal, fill form" workflow with a single line typed from anywhere.

```
/novoDeal Joao da Silva - Video Institucional
```

Bot: "Projeto criado! ID: #42 - Joao da Silva (Video Institucional). Status: Novo."

B. /briefing - Register a Briefing

After a client meeting or call, the team member types `/briefing` with the project ID and a free-text description of what the client wants. The backend stores this in a dedicated briefings table, linked to the Deal. This ensures that creative requirements, references, and client expectations are captured immediately rather than relying on memory or scattered notes. The briefing text is searchable from the dashboard.

```
/briefing #42 O cliente quer video de 1 minuto focado em vendas para
Instagram. Referencia: [link]
```

Bot: "Briefing salvo para o projeto #42!"

C. /despesa - Register an Expense

Filmmakers and photographers have project-specific expenses: equipment rental, location fees, freelance crew, props, and more. The `/despesa` command lets the team log these expenses in real-time, tied to a specific Deal. The backend creates a record in the financial_transactions table with the amount, description, and project link. The dashboard shows a running total of expenses per project, enabling profitability tracking without switching to a separate accounting tool.

```
/despesa #42 Aluguel de camera R$500
```

Bot: "Despesa de R$500 registrada no projeto #42 (Aluguel de camera)."

D. /status - Quick Project Status

The `/status` command gives the team an instant snapshot of a project. The backend queries the deals, briefings, financial_transactions, bookings, and documents tables to compile a summary: client name, current status, whether a briefing exists, whether a quote has been sent, total expenses, and upcoming dates. This is the "at-a-glance" view that a filmmaker needs before walking into a client meeting or when reviewing the week ahead.

```
/status #42
```

Bot: "Projeto #42: Joao da Silva. Status: Briefing OK. Orcamento: Pendente. Despesas: R$500."

3. Architecture

The system has three layers: the WhatsApp interface (bot), the backend (Node.js + Supabase), and the frontend dashboard (Lovable for MVP, Next.js for production). All data lives in Supabase PostgreSQL, which acts as the single source of truth. The WhatsApp bot sends webhooks to a Node.js server (deployed on Vercel or Railway), which processes messages, stores data, and triggers outbound messages. The dashboard reads from and writes to the same Supabase instance.

Component	Technology	Role
WhatsApp	Cloud API (direct)	Official API, no BSP markup, full document/interactive support
Webhook server	Node.js + Express (TypeScript)	Receives webhooks, processes messages, sends responses
Database	Supabase PostgreSQL	Clients, deals, briefings, expenses, bookings, documents
File storage	Supabase Storage	PDFs, contracts, quotes, invoices

Edge Functions	Supabase Edge Functions (Deno)	PDF generation, Cal.com sync, outbound message triggers
Calendar	Cal.com API	Booking creation, availability, confirmations
Dashboard (MVP)	Lovable + Supabase	CRM interface, calendar, document generator, financial view
Dashboard (Prod)	Next.js + shadcn/ui + Supabase	Full control, custom calendar, advanced analytics, multi-role
Deployment	Vercel (serverless) + Supabase Cloud	Zero DevOps, free tiers cover MVP

Table 3: System Architecture

3.1 Data Flow: Client Sends Info

Client messages WhatsApp, the Cloud API sends a webhook to the Node.js server, which parses the message and identifies the intent (new booking, quote request, etc.). The server writes client data and the message to Supabase, then sends the next prompt or confirmation via the Cloud API. The dashboard updates in real-time via Supabase Realtime. If the client completes a booking flow, the server also calls the Cal.com API to create a pending event, which appears in the dashboard calendar.

3.2 Data Flow: Team Uses Slash Commands

A team member types a slash command (e.g., `/novoDeal`, `/briefing`, `/despesa`, `/status`) in the WhatsApp group or direct chat. The Node.js server receives the webhook, parses the command and arguments using a simple regex-based command router, validates the input (project ID exists, amount is a number, etc.), and writes to the appropriate Supabase table. The server then sends a confirmation message back to WhatsApp. The dashboard reflects the new data instantly via Realtime subscriptions. This flow is synchronous from the user perspective: type a command, get a confirmation within 1-2 seconds.

3.3 Data Flow: Pro Sends PDF

The professional clicks "Send Contract" on a client page in the dashboard. The dashboard calls a Supabase Edge Function with template type and client ID. The Edge Function generates the PDF using template data, uploads it to Supabase Storage, then calls the Cloud API to send a document message with the PDF URL. The client receives the PDF in WhatsApp. Message status (delivered, read) is tracked via webhook and updated in the dashboard.

4. Database Schema

The schema is designed for the filmmaker/photographer use case, with tables for clients, deals (the central entity), briefings, financial transactions, conversations, messages, bookings, documents, and templates. All tables use UUIDs as primary keys and timestamps for auditing. The "deals" table is the hub: briefings, expenses, documents, and bookings all link back to a deal. This mirrors how a filmmaker thinks about their work: everything revolves around a project.

Table	Key Columns	Purpose
clients	id, phone, name, email, event_type, notes, source, created_at	Client records, auto-populated from WhatsApp intake
deals	id, client_id (FK), title, status (novo/briefing/contoando/producao/finalizado), value, created_at	Central project entity, created via /novoDeal or client intake
briefings	id, deal_id (FK), content, author, created_at	Creative briefings captured via /briefing command
financial_transactions	id, deal_id (FK), type (receita/despesa), amount, description, created_at	Income and expenses tracked via /despesa and dashboard
conversations	id, client_id (FK), status, created_at	Conversation threads, one per client
messages	id, conversation_id (FK), direction, type, content, status, created_at	All messages (inbound + outbound)
bookings	id, deal_id (FK), client_id (FK), event_date, event_type, location, status, cal_com_event_id	Shoot bookings linked to Cal.com
documents	id, deal_id (FK), client_id (FK), type (contract/quote/invoice), storage_url, sent_at, status	PDFs generated and sent via WhatsApp
templates	id, type, title, body_html, variables, created_at	Document templates for PDF generation

Table 4: Core Database Schema

5. WhatsApp Message Flows

5.1 New Client Intake

Step	Bot Action	WhatsApp Feature
1	Greet and offer options	Quick Reply Buttons (Book Shoot / Get Quote / Talk to Us)
2	Ask event type	List Message (Wedding, Corporate, Portrait, Product, Other)
3	Ask preferred date	Text input (free-form date)
4	Ask location	Text input or Location message
5	Ask special requests	Text input (optional, skip button available)
6	Confirm all details	Quick Reply Buttons (Confirm / Edit)
7	Save to DB + create deal	Text confirmation with next steps

Table 5: New Client Intake Flow

5.2 Quote / Contract Delivery

Step	Trigger	Action
1	Pro clicks "Send Quote" in dashboard	Edge Function generates PDF, uploads to Supabase Storage
2	PDF ready	Cloud API sends document message with PDF + caption
3	Client receives PDF	Webhook updates delivery status in dashboard
4	Client replies with questions	Bot forwards to dashboard as notification

Table 6: Quote/Contract Delivery Flow

5.3 Booking Confirmation

When the professional confirms a booking in the dashboard, the system sends a WhatsApp template message to the client with the confirmed date, time, and location. Template messages are required here because the confirmation is business-initiated (outside the 24-hour service window). The template includes a CTA URL button linking to the Cal.com booking page for the client to add the event to their own calendar. If the client needs to reschedule, they reply to the message (which re-opens the 24-hour window for free-form messaging) and the bot offers available alternative dates.

5.4 Slash Command Flows

Slash commands follow a consistent pattern: the server receives the message, a command router identifies the slash prefix, extracts arguments using regex, validates them, performs the database operation, and returns a human-readable confirmation. Error handling includes: unknown project ID, missing arguments, and invalid amount formats. The command router is a simple switch statement in the Express.js webhook handler, making it trivial to add new commands over time.

Command	Arguments	DB Operation	Response
/novoDeal	Name - Project Type	INSERT clients + deals	Project ID + status
/briefing	#ID Free text	INSERT briefings (deal_id)	Confirmation
/despesa	#ID Description R$Amount	INSERT financial_transactions (deal_id)	Amount + project
/status	#ID	SELECT deals + briefings + financial_transactions	Full project summary

Table 7: Slash Command Reference

6. Costs and Setup Requirements

6.1 WhatsApp Cloud API Costs

Meta charges per message based on category. For a filmmaker/photographer CRM, most messages fall into the Utility or Service category, which are the cheapest or free. The 24-hour service window is your best friend: when a client messages you, all replies within 24 hours are free. You only pay when you initiate a conversation outside that window (e.g., sending a contract 3 days after the last client message). Template messages for bookings and quotes cost approximately $0.005-0.01 per message in the Utility category.

Scenario	Cost	Notes
Client initiates conversation	FREE	24h service window opens
Your replies within 24h	FREE	Unlimited free replies
Contract/quote sent after 24h	~$0.01/msg	Utility template message
Booking confirmation	~$0.01/msg	Utility template message
Marketing follow-up	~$0.02/msg	Marketing template (avoid if possible)

Table 8: Per-Message Cost Estimates

6.2 Infrastructure Costs

Service	Free Tier	Paid (if needed)
Supabase	500 MB DB, 1 GB storage, 50K MAU	$25/mo (Pro)
Vercel (Node.js)	100K serverless invocations/mo	$20/mo (Pro)
Cal.com	1 event type, 100 bookings/mo	$12/mo (Teams)
WhatsApp Cloud API	No platform fee, only per-message	Per-message only
Lovable (MVP)	Free tier available	$20/mo (Pro)
Next.js (Production)	Deployed on Vercel, same free tier	$20/mo (Pro)

Table 9: Infrastructure Cost Breakdown

MVP total: $0-10/month (Supabase free tier + Vercel hobby + Cal.com free tier + WhatsApp per-message fees only). The system can comfortably serve 50-100 clients per month on free tiers. Production with Next.js adds no extra hosting cost since it also deploys on Vercel.

6.3 Meta Business Account Setup

To use the Cloud API, you need: (1) a Meta Business Manager account at business.facebook.com, (2) a verified business phone number (cannot be currently active on WhatsApp mobile), (3) a Facebook app with WhatsApp Business API product added at developers.facebook.com, (4) business verification (documents like business license or utility bill). The verification process takes 24-72 hours. During development, Meta provides a test phone number with free messaging to verified test contacts.

7. Implementation Plan

Week	Milestone	Deliverables
1	Cloud API + webhook	Meta Business account, webhook server on Vercel, verify webhook receives messages

2	Client intake bot	Interactive message flows (buttons, lists), Supabase schema, client + deal record creation
3	Slash commands	Command router (/novoDeal, /briefing, /despesa, /status), Supabase writes, confirmation messages
4	PDF generation + delivery	Supabase Edge Function for PDF, storage upload, document message sending
5-6	Dashboard MVP (Lovable)	Lovable app: client list, deal detail, send PDF button, basic calendar, expense view
7-8	Calendar integration	Cal.com API setup, booking creation from WhatsApp, confirm/reschedule flow
9-10	Production dashboard (Next.js)	Next.js app with shadcn/ui: advanced calendar, financial reports, multi-role auth
11-12	Polish + deploy	Template messages, error handling, notifications, production deployment

Table 10: Implementation Timeline

The first 3 weeks give you a working bot where clients can submit info and the team can use slash commands. Weeks 4-8 add PDF delivery, the Lovable dashboard, and calendar integration. Weeks 9-12 migrate to Next.js and harden for production. Each week assumes part-time effort (10-15 hours). Full-time, cut the timeline in half.

8. Dashboard Alternatives

8.1 Why Next.js Was Not the First Choice for MVP

Next.js was not recommended as the MVP dashboard for a simple reason: speed to first working version. Lovable generates a functional CRUD dashboard connected to Supabase in hours, not weeks. For a solo JavaScript developer validating a product idea, this is the difference between shipping in 2 weeks and shipping in 6 weeks. Lovable handles UI layout, state management, and Supabase integration with AI-assisted prompting, so you focus on the bot logic and data model rather than building UI components from scratch.

However, Lovable has real limitations that make it unsuitable as a long-term production dashboard. The sections below detail those limitations and explain why Next.js is the recommended path for the production version.

8.2 Lovable Limitations (Why You Will Outgrow It)

1. Limited backend control. Lovable generates frontend code that calls Supabase directly. Complex business logic (e.g., calculating project profitability with expense aggregation, sending WhatsApp messages from the dashboard, generating PDFs with custom templates) cannot live in Lovable. You need Supabase Edge Functions or a separate API layer, which creates a split-brain architecture where some features are in Lovable and others are in serverless functions.

2. Edge Function time limits. Supabase Edge Functions (Deno) have a 150-second timeout on the free tier and 400 seconds on Pro. PDF generation for complex documents with images can exceed these limits. A Next.js API route running on Vercel has no such constraint for most use cases, and you can always fall back to a long-running worker on Railway or Fly.io.

3. Vendor lock-in. Lovable generates code that is difficult to eject. The AI-generated components use Lovable-specific patterns and abstractions that do not map cleanly to standard React. Migrating away from Lovable often means rewriting the frontend rather than refactoring it. Next.js with shadcn/ui, by contrast, produces standard React code that you own and can deploy anywhere.

4. Calendar and scheduling UI. Lovable does not have a native calendar component. Building a drag-and-drop calendar view (day/week/month) with Cal.com integration requires custom React components, which defeats the purpose of using a low-code tool. Next.js + a library like react-big-calendar or FullCalendar gives you a production-grade calendar in a day of work.

5. Multi-role access control. As your studio grows, you will need different permission levels: admin, producer, editor, and client portal. Lovable does not natively support role-based access control (RBAC). Next.js with NextAuth.js or Supabase Auth gives you fine-grained RBAC out of the box, including row-level security in Supabase that restricts data access per role.

6. Custom analytics and reports. Filmmakers need project profitability reports, monthly revenue dashboards, and expense breakdowns by category. These require complex SQL queries and chart libraries (Recharts, Nivo, or Chart.js). Lovable can display basic tables, but building interactive charts with filters and date ranges requires the control that Next.js provides.

8.3 Next.js for Production: Pros and Cons

Aspect	Pros	Cons
Control	Full ownership of code, deploy anywhere, no vendor lock-in	More development time, need to build all components
Calendar UI	react-big-calendar, FullCalendar, drag-and-drop scheduling	Need to integrate and configure calendar library
Auth/RBAC	NextAuth.js + Supabase Auth + row-level security	Need to design permission model and implement guards

Analytics	Recharts, Nivo, Chart.js for custom reports and dashboards	Need to build chart components and data aggregation layer
PDF preview	In-browser PDF viewer with annotation support	Need to integrate react-pdf or similar library
Performance	SSR/SSG for fast page loads, API routes for backend logic	More complex deployment model than Lovable
WhatsApp from UI	Direct API calls from Next.js API routes, no Edge Function limits	Need to manage Cloud API tokens securely
Cost	Same Vercel hosting, no additional platform fee	More developer hours = higher opportunity cost early on

Table 11: Next.js Production Dashboard - Pros and Cons

8.4 Other Alternatives

Tool	Strengths	Weaknesses	Fit
Lovable	Fastest UI build, native Supabase, AI-assisted dev	Limited backend control, vendor lock-in, no calendar component	Best for MVP
Bolt.new	Similar to Lovable, good Supabase support, faster iteration	Newer platform, smaller community, similar limitations	Good alternative
Next.js + shadcn/ui	Full control, no vendor lock-in, unlimited customization	More development time, need to build all components	Best for scale
Retool	Built-in database connectors, low-code, fast internal tools	Not great for external-facing UIs, pricing at scale	Internal tool only
Appsmith	Open-source, self-hostable, good for CRUD dashboards	Heavier setup, UI less polished than Lovable	Self-host option

Table 12: Dashboard Tool Comparison

8.5 Recommended Migration Path

Start with Lovable for the MVP (weeks 1-8). It will get you a working dashboard in days rather than weeks. In parallel, begin building the Next.js app (weeks 9-12) with shadcn/ui components connected to the same Supabase backend. Since the database and backend are independent of the frontend, migration

is straightforward: build the Next.js app alongside Lovable, switch over when ready, and retire the Lovable version. The key Next.js components to build first are: the calendar view (which Lovable cannot do well), the financial dashboard (which requires charts), and the WhatsApp message panel (which requires direct Cloud API integration).

9. Key Technical Decisions

9.1 Why Cloud API Over Unofficial Libraries

For a client-facing CRM, reliability and compliance are non-negotiable. Unofficial libraries like Baileys work for prototypes but carry ban risk and cannot send template messages (required for business-initiated conversations outside the 24h window). The Cloud API is the only way to send PDFs as proper document messages with delivery tracking, and the only way to use interactive message types officially. The per-message cost for a photographer with 50 clients/month is under $5.

9.2 Why Supabase Over MongoDB

Supabase gives you PostgreSQL with real-time subscriptions (critical for the dashboard to update when new WhatsApp messages arrive), built-in auth (if you later add team members), file storage for PDFs, and edge functions for serverless PDF generation. MongoDB is a fine database, but Supabase provides the entire backend stack (database + auth + storage + functions + realtime) with native Lovable integration. For a solo JS developer, this means writing almost zero backend infrastructure code.

9.3 Why Cal.com Over Custom Calendar

Cal.com handles the hardest parts of scheduling: timezone conversion (essential when clients book from different cities), availability management, conflict detection, and client-facing booking pages. Building these features from scratch in Supabase would add 2-3 weeks to the project. Cal.com's free tier covers 1 event type and 100 bookings/month, which is sufficient for most solo photographers. If you need multiple event types (wedding, portrait, corporate), the $12/month Teams plan removes this limit.

9.4 Why Slash Commands Over a Separate App

The slash command approach keeps the team inside WhatsApp, which is already their primary communication tool. Filmmakers on set do not want to switch between WhatsApp (for client comms) and a separate app (for CRM data entry). By embedding CRM operations inside WhatsApp itself, you eliminate friction and increase adoption. The tradeoff is that slash commands are limited in expressiveness: you cannot upload images, select from dropdowns, or fill multi-step forms. For those, the team uses the dashboard. The commands are designed for the 80% case: quick data capture that takes less than 10 seconds to type.

10. Next Steps

To start building this week, follow these steps in order:

1. Create a Meta Business Manager account at business.facebook.com
2. Create a Facebook app at developers.facebook.com and add the WhatsApp product
3. Set up a Supabase project and run the database schema (Table 4)
4. Build the Express.js webhook server with command router and deploy to Vercel
5. Test the webhook with Meta's test phone number (free messaging to test contacts)
6. Implement the client intake flow (Section 5.1)
7. Implement slash commands: /novoDeal, /briefing, /despesa, /status (Section 5.4)
8. Build the Lovable dashboard connected to the same Supabase project
9. Add PDF generation and delivery (Section 5.2)
10. Integrate Cal.com for calendar (Section 2.3)
11. Apply for template message approval for contract/quote/booking templates
12. Begin Next.js production dashboard build (Section 8.3)

Steps 1-6 can be completed in a single weekend. By the end of week 2, you will have a bot that collects client info and accepts slash commands. By the end of week 4, you will have a full MVP with dashboard, PDF delivery, and scheduling. By week 12, you will have a production-ready Next.js dashboard with advanced features.